

Livro-texto

Computer **Organization** and Design: The Hardware/Software Interface, **5th edition**

- Autores: David Patterson and John Hennessy
- **MIPS edition** (Não ARM edition! Nem RISC V edition!)
- Exemplos na BU: **4th edition**



Introdução à programação de sistemas

Introdução

- Codificação **binária** de instruções
 - Natural e eficiente para os computadores
 - » Números binários
- Codificação **simbólica** de instruções
 - Mais apropriada aos seres humanos
 - » Mnemônicos

Representações de código: linguagem de máquina

```
001001111011110111111111111100000
1010111110111111100000000000010100
101011111010010000000000000100000
101011111010010100000000000100100
10101111101000000000000000011000
10101111101000000000000000011100
10001111101011100000000000011100
10001111101110000000000000011000
00000001110011100000000000011001
00100101110010000000000000000001
00101001000000010000000001100101
10101111101010000000000000011100
00000000000000000111100000010010
00000011000011111100100000100001
00010100001000001111111111110111
10101111101110010000000000011000
00111100000001000001000000000000
10001111101001010000000000011000
000011000001000000000000011101100
00100100100001000000010000110000
10001111101111111000000000010100
001001111011111010000000000100000
0000001111100000000000000001000
00000000000000000001000000100001
```

Calcula e imprime a soma dos quadrados dos inteiros de 0 a 100

Representações de código: linguagem de montagem

Operação

```
addiu    $29, $29, -32
sw        $31, 20($29)
sw        $4, 32($29)
sw        $5, 36($29)
sw        $0, 24($29)
sw        $0, 28($29)
lw        $14, 28($29)
lw        $24, 24($29)
multu    $14, $14
addiu    $8, $14, 1
slti      $1, $8, 101
sw        $8, 28($29)
mflo      $15
addu      $25, $24, $15
bne       $1, $0, -9
sw        $25, 24($29)
lui       $4, 4096
lw        $5, 24($29)
jal        1048812
addiu     $4, $4, 1072
lw        $31, 20($29)
addiu     $29, $29, 32
jr        $31
move      $2, $0
```

Registrador
Constante

Calcula e imprime a soma dos quadrados dos inteiros de 0 a 100

Representações de código: linguagem de montagem

```
.text
.align    2
.globl    main

main:
    subu    $sp, $sp, 32
    sw      $ra, 20($sp)
    sd      $a0, 32($sp)
    sw      $0, 24($sp)
    sw      $0, 28($sp)

loop:
    lw      $t6, 28($sp)
    mul     $t7, $t6, $t6
    lw      $t8, 24($sp)
    addu    $t9, $t8, $t7
    sw      $t9, 24($sp)
    addu    $t0, $t6, 1
    sw      $t0, 28($sp)
    ble     $t0, 100, loop
    la      $a0, str
    lw      $a1, 24($sp)
    jal     printf
    move    $v0, $0
    lw      $ra, 20($sp)
    addu    $sp, $sp, 32
    jr      $ra

.data
.align    0
str:
.asciiz    "The sum from 0 .. 100 is %d\n"
```

Diretivas

Registrador

Representações de código: linguagem de montagem

```
.text
.align 2
.globl main
main:
    subu    $sp, $sp, 32
    sw      $ra, 20($sp)
    sd      $a0, 32($sp)
    sw      $0, 24($sp)
    sw      $0, 28($sp)
loop:
    lw      $t6, 28($sp)
    mul     $t7, $t6, $t6
    lw      $t8, 24($sp)
    addu    $t9, $t8, $t7
    sw      $t9, 24($sp)
    addu    $t0, $t6, 1
    sw      $t0, 28($sp)
    ble     $t0, 100, loop
    la      $a0, str
    lw      $a1, 24($sp)
    jal     printf
    move    $v0, $0
    lw      $ra, 20($sp)
    addu    $sp, $sp, 32
    jr      $ra

.data
.align 0
str:
.asciiz "The sum from 0 .. 100 is %d\n"
```

Labels

Representações de código: linguagem de alto nível

```
#include <stdio.h>

int
main (int argc, char *argv[])
{
    int i;
    int sum = 0;
    for (i = 0; i <= 100; i = i + 1) sum = sum + i * i;
    printf ("The sum from 0 .. 100 is %d\n", sum);
}
```

Rotina de biblioteca

Variável **Constante** **Operação**

Calcula e imprime a soma dos quadrados dos inteiros de 0 a 100

A noção de montagem

- Linguagem de máquina (“**machine** language”)
 - Representação binária das instruções de um processador
- Linguagem de montagem (“**assembly** language”)
 - Representação simbólica da linguagem de máquina

A noção de montagem

- Linguagem de máquina (“**machine** language”)
 - Representação binária das instruções de um processador
 - Instruções divididas em campos codificados em binário
 - » Exemplo: 000000 10001 10010 01000 00000 100000
- Linguagem de montagem (“**assembly** language”)
 - Representação simbólica da linguagem de máquina

A noção de montagem

- Linguagem de máquina (“**machine** language”)
 - Representação binária das instruções de um processador
 - Instruções divididas em campos codificados em binário
 - » Exemplo: 000000 10001 10010 01000 00000 100000
- Linguagem de montagem (“**assembly** language”)
 - Representação simbólica da linguagem de máquina
 - Mnemônicos associados a campos da instrução binária
 - » Exemplo: add \$t0, \$s1, \$s2

Elementos do “assembly”

- **Mnemônicos de operações**
 - Exemplos: add, sub, jal
- **Mnemônicos de registradores**
 - Exemplo: \$s1
- **Mnemônicos de endereços de memória**
 - Rótulos (“**Labels**”)
 - Exemplo:
main: add \$t0, \$s1, \$s2

Elementos do “assembly”

- **Pseudo-instruções**
 - São representadas como se fossem instruções
 - Não são nativas do processador
 - Representação alternativa ou condensada
- **Exemplo:**
 - `move $t0, $t1    # $t0 ← $t1`
 - `add $t0, $zero, $t1    # $t0 ← $t1 + 0`

O programa montador

- Tradução operações e registradores

- Mnemônico → número(s) binário(s)

add \$t0, \$s1, \$s2 → 000000 10001 10010 01000 00000 100000

- Tradução de rótulos (“labels”)

- Endereço simbólico → endereço binário

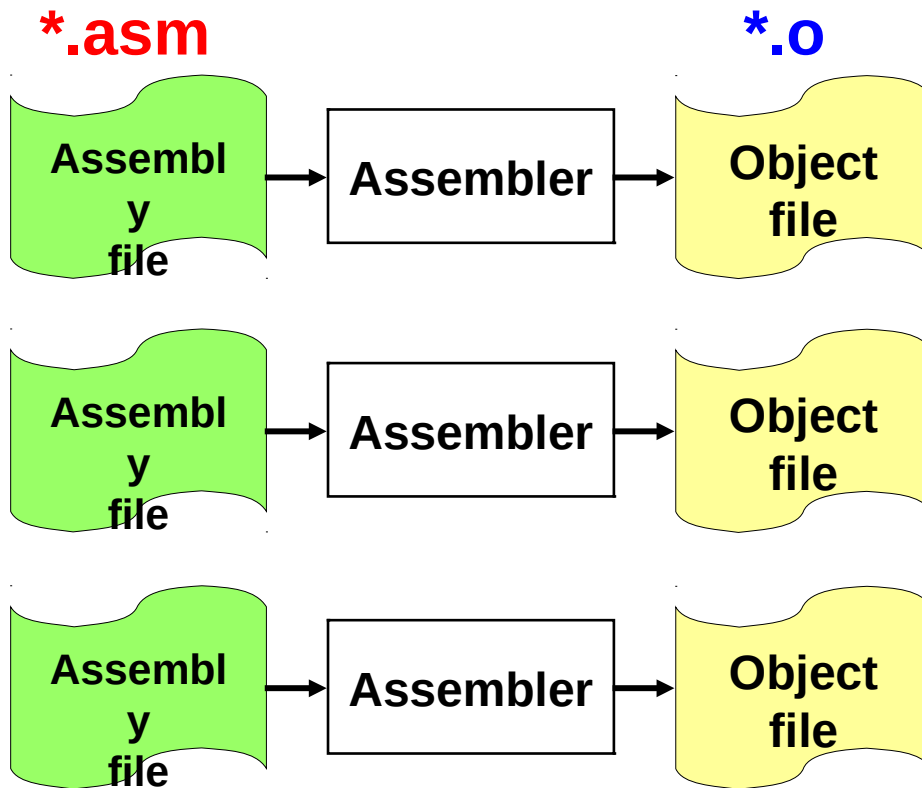
jal main → 000011 1010101010101010101010 101010

- Tradução de pseudo-instruções

- Pseudo-instrução → instrução nativa

move \$t0, \$t1 → **add \$t0, \$zero, \$t1**

O programa montador



Gera um arquivo **objeto** para cada módulo

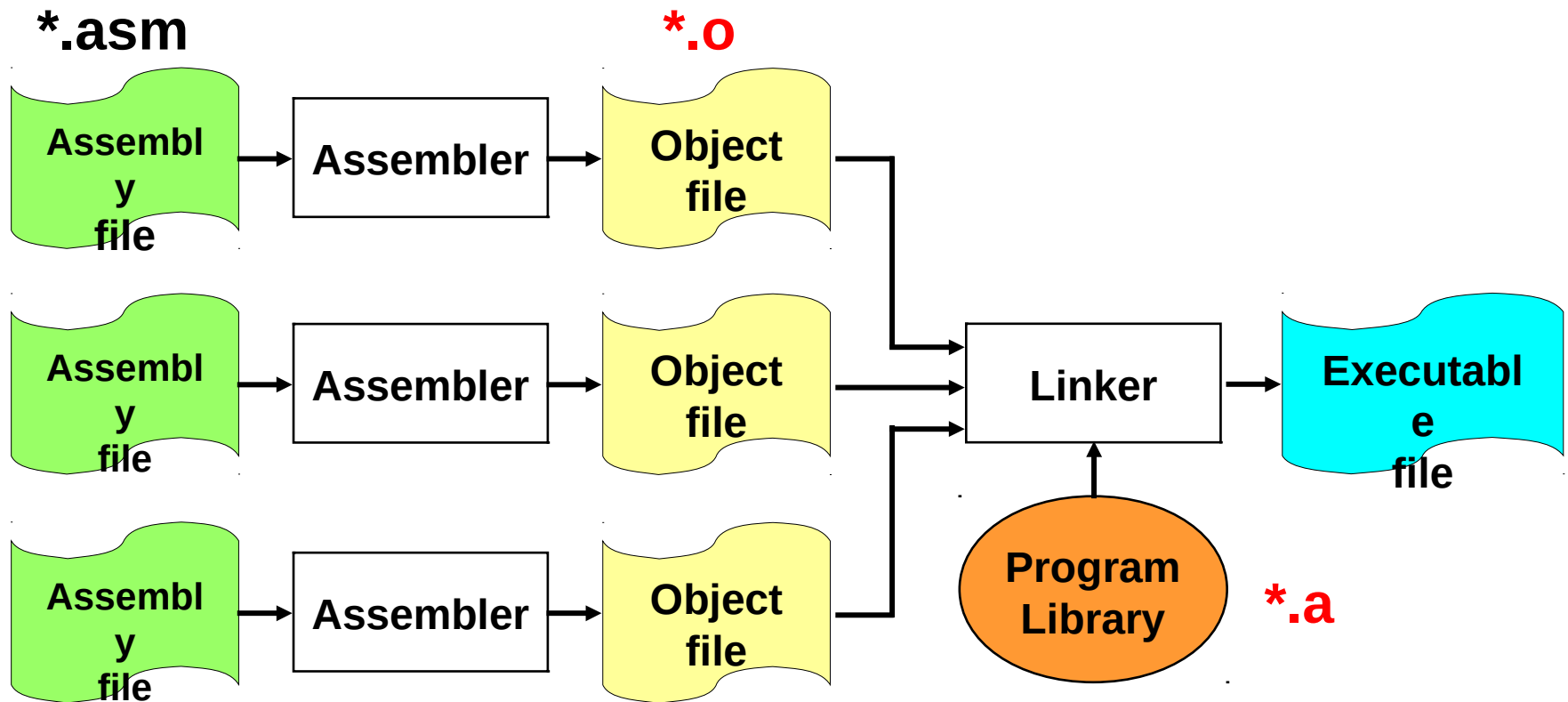
A noção de ligação

- **Programas são divididos em módulos**
 - Desenvolvidos independentemente
 - Compilados e montados independentemente
- **Programas usam bibliotecas**
 - Rotinas pré-desenvolvidas
 - Exemplo: “standard C library”

A noção de ligação

- Módulo contém **referências**
 - Labels de subrotinas e dados
 - Definidos em outro módulo ou biblioteca
- Referências não resolvidas
 - Código do módulo não pode ser executado
- É preciso editar as referências
 - Para combinar arquivos-objeto e biblioteca
 - Gerando um único **arquivo executável**

O programa ligador



Gera o arquivo **executável**

Uso do “assembly”

- **Tamanho/Velocidade do programa é crítico**
 - **Computador embarcado (“embedded computer”)**
 - **“High end”:**
 - » **Exemplos: controle de freios (ABS) e estabilidade (ESC)**
 - » **Objetivo: Viabilização de restrição temporal**
 - **Resposta rápida e previsível**
 - **Garantia de resposta em tempo determinado**
 - **“Low-end”**
 - » **Exemplo: controle de eletrodomésticos (temporizador programável)**
 - » **Objetivo: redução de custo de produto**
 - **Menos instruções, menor quantidade de memória**

Uso do “assembly”

- **Trechos críticos de um programa**
 - **Deteção de trechos críticos**
 - » Maior parte do tempo em pequenos trechos de código
 - » Ferramentas de “profiling”
 - **Recodificação em alto nível**
 - » Melhores algoritmos e estruturas de dados
 - **Recodificação em assembly**
 - » Permite maior desempenho
 - **Enfoque híbrido:**
 - » Maior parte em linguagem de alto nível: **C, C++**
 - » Trecho crítico em “assembly”: **asm("move \$t0, \$t1");**

Uso do “assembly”

- **Não disponibilidade de um compilador**
 - **“Low-end”:**
 - » **Microcontroladores de baixíssimo custo**
 - Exemplo: alguns usados em controle de telefones sem fio
 - **“High-end”:**
 - » **Processadores dedicados à aplicação específica**
 - » **ASIPs : “Application-specific instruction-set processor”**
 - Exemplo: áudio digital

Vantagens do “assembly”

- **Programador controla melhor os recursos do hardware**
 - **Pode gastar mais tempo e criatividade otimizando**
 - **Compiladores produzem código mais uniforme**
 - » **Mas não enxergam todas as otimizações possíveis**

Vantagens do “assembly”

- **Exploração de instruções especializadas**
 - **Exemplos:**
 - » **Cópia de strings**
 - » **Instruções multimedia**
 - » **Laços em instrução única**
 - » **Instruções recentemente criadas para estender uma ISA, mas ainda não exploradas em compiladores distribuídos em domínio público (ex. **instruções AVX**)**

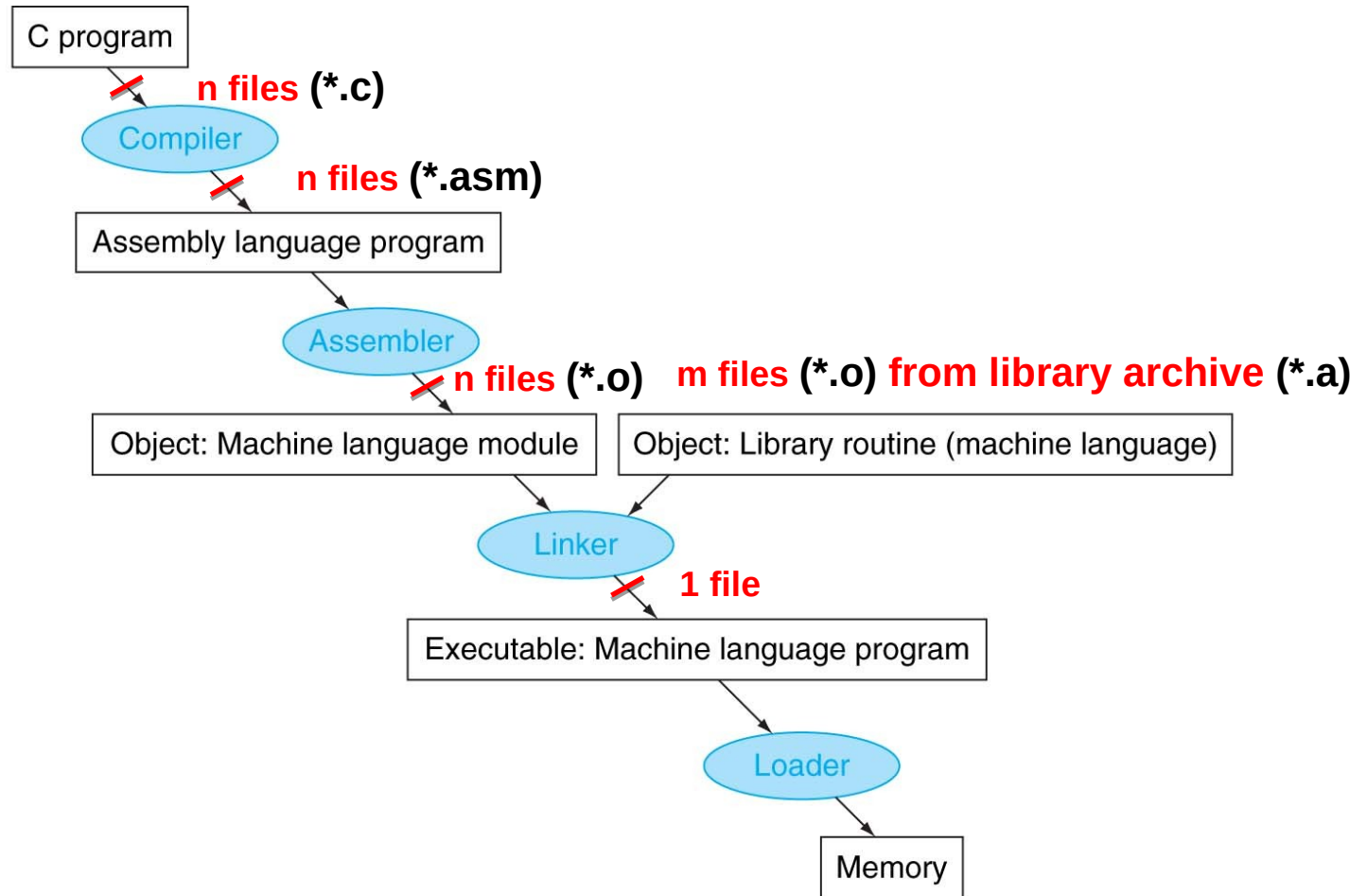
Desvantagens do “assembly”

- **Não portabilidade**
 - **Código reescrito para cada novo processador**
 - » Só pode ser usado na mesma família (ISA)
- **Programas mais longos**
 - **Exemplo desta aula:**
 - » C (15 linhas) x assembly (31 linhas)
 - **Programas mais complexos: maior expansão**

Desvantagens do “assembly”

- **Menor produtividade**
 - Programadores escrevem por dia o mesmo número de linhas de código
 - » Em alto nível ou em assembly
 - Se programa expande n vezes
 - » Produtividade em alto nível é n vezes maior
- **Menor legibilidade**
 - Mais difícil de ler e depurar
 - Falta de estrutura (ex. “if-then-else”)
 - Análise requer “reconstrução” do algoritmo

Cadeia de ferramentas



Introdução à programação de sistemas

Top Languages

- **IEEE Spectrum 2022 ranking**
 - <https://spectrum.ieee.org/top-programming-languages-2022>
 - **Metrics from 8 sources:**
 - » IEEE JobSite, CarrerBuillder
 - » Google, Github, IEEE Digital Library
 - » StackOverflow, Reddit, Twitter
- **Rankings diferentes com maiores pesos para:**
 - Interesses de membros da IEEE = **Spectrum**
 - Anúncios de empregos = **Jobs**
 - Redes sociais e forums = **Trending**

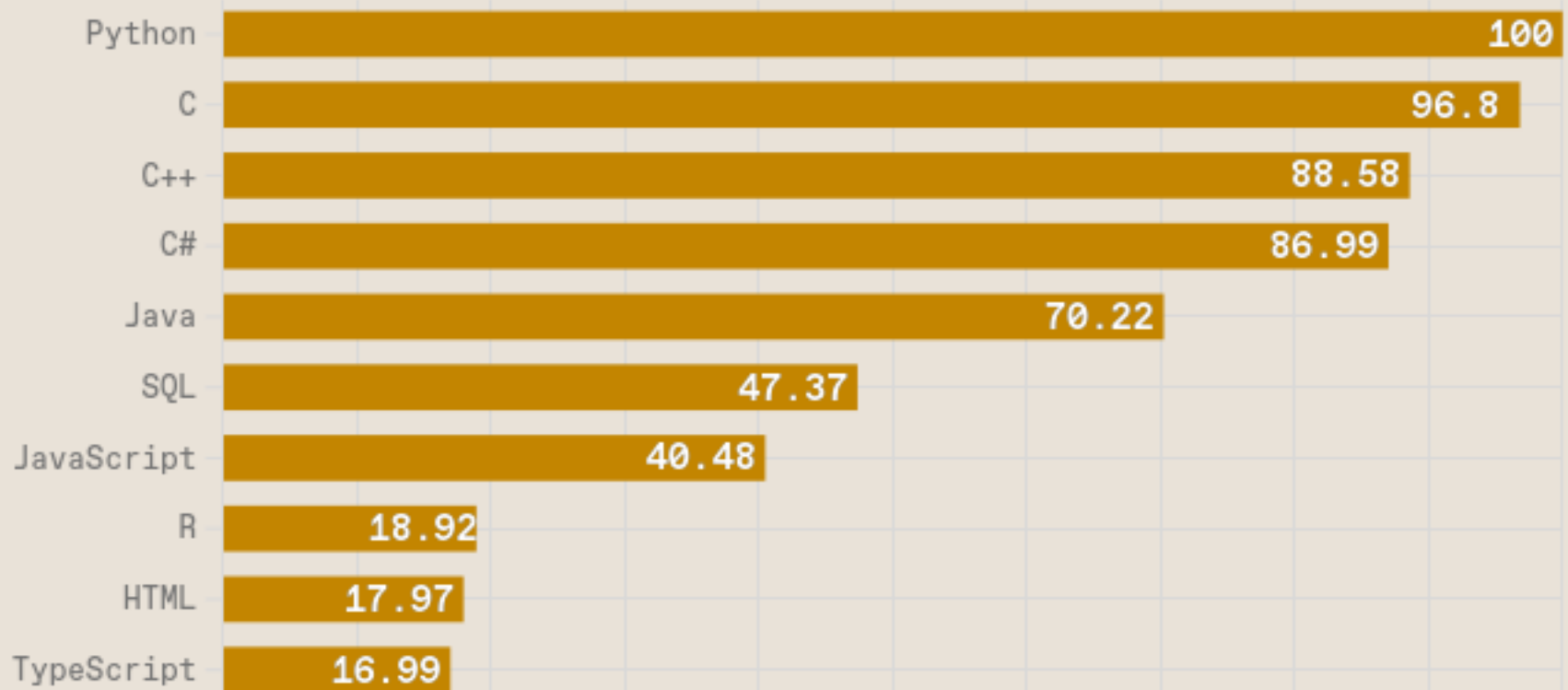
Top Programming Languages 2022

Click a button to see a differently weighted ranking

Spectrum

Jobs

Trending

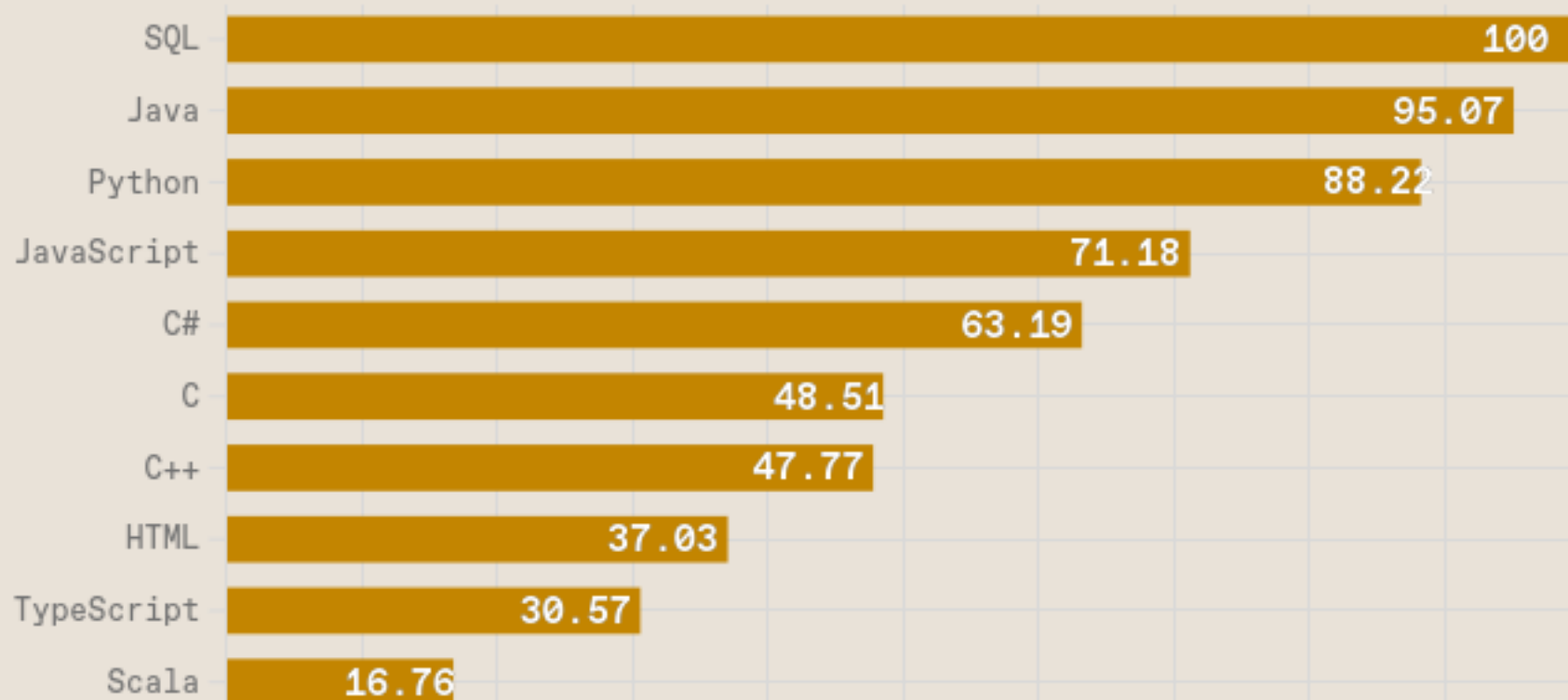


Python (produtividade, bibliotecas ML/SC) vs C (velocidade, embed/HPC)

Top Programming Languages 2022

Click a button to see a differently weighted ranking

Spectrum **Jobs** Trending



SQL (consulta e manipulação de banco de dados)

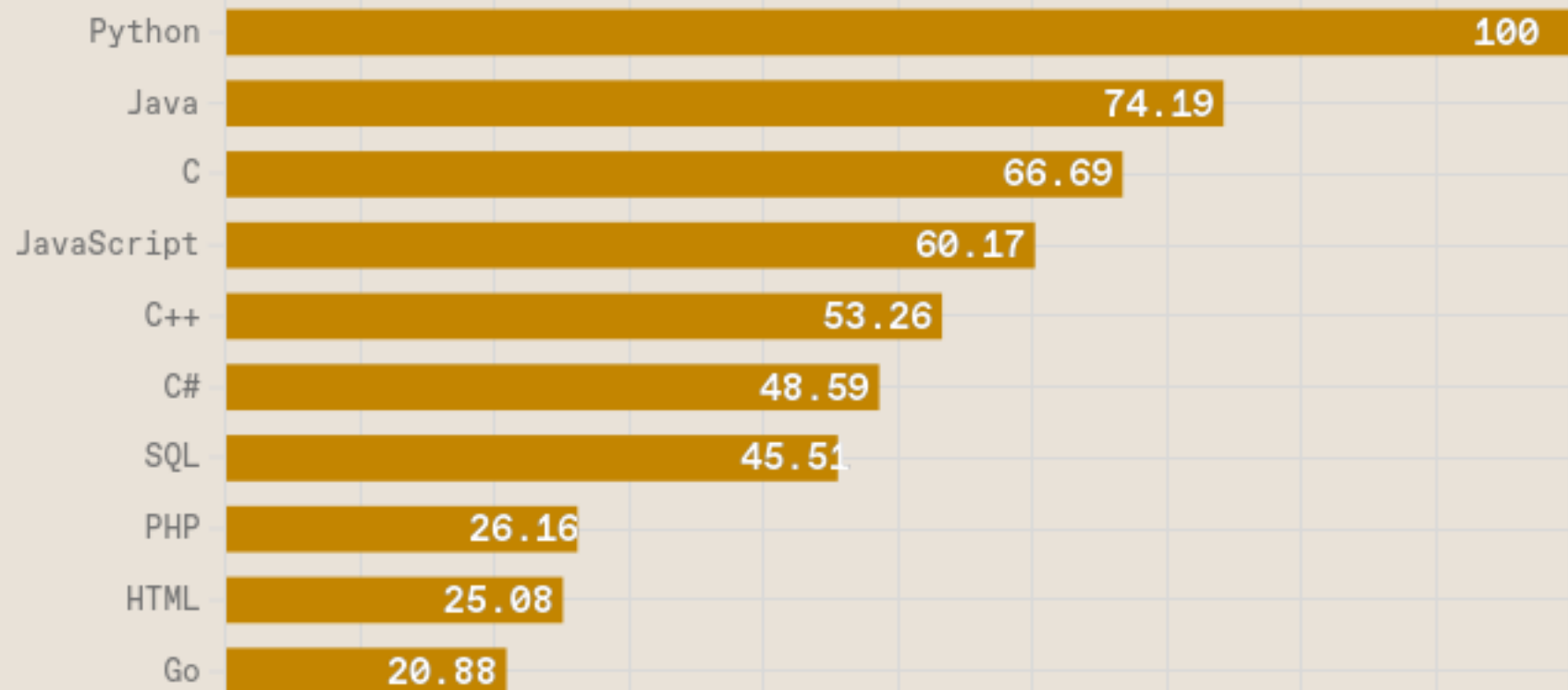
Top Programming Languages 2022

Click a button to see a differently weighted ranking

Spectrum

Jobs

Trending



Java e Java script (aplicações web)

Assembly como linguagem

- **Spectrum definition:** A catch call term for a vast **family of processor instruction sets**, writing assembly code requires **considerable expertise**, but it allows the creation of **high-speed software** that can run directly “on the metal”
- **Ranking** (close do Verilog and VHDL)
 - Spectrum: 5.73
 - Jobs: 1.02
 - Trending: 11.56